Lab Sheet 1: DDL Commands

CREATE, ALTER and DROP — SQL Answers with Expected Outputs

SQL dialect used: MySQL 8.x. The outputs below are representative console outputs; exact wording/row counts can vary slightly by MySQL version or client (MySQL Workbench, phpMyAdmin, terminal, etc.). Tasks that reuse the same table name are shown as independent demonstrations and may require resetting/recreating the table before execution.

1. Create a database named CollegeDB.

SQL Answer:

```
CREATE DATABASE CollegeDB;
USE CollegeDB;
```

Expected Output:

```
Query OK, 1 row affected

Database changed
```

2. Create a Student table with Student_ID, Name, Age, Course and City.

SQL Answer:

```
CREATE TABLE Student (
    Student_ID INT,
    Name VARCHAR(100),
    Age INT,
    Course VARCHAR(100),
    City VARCHAR(100)
);
```

Expected Output:

```
Query OK, 0 rows affected
```

3. Create an Employee table with Emp_ID, Emp_Name, Department and Salary.

SQL Answer:

```
CREATE TABLE Employee (
    Emp_ID INT,
    Emp_Name VARCHAR(100),
    Department VARCHAR(100),
    Salary DECIMAL(10,2)
);
```

Expected Output:

```
Query OK, 0 rows affected
```

4. Create a Department table with Dept_ID, Dept_Name and Location.

SQL Answer:

```
CREATE TABLE Department (
    Dept_ID INT,
    Dept_Name VARCHAR(100),
    Location VARCHAR(100)
);
```

Expected Output:

```
Query OK, 0 rows affected
```

5. Create a Product table with Product_ID, Product_Name, Price and Quantity.

SQL Answer:

```
CREATE TABLE Product (
    Product_ID INT,
    Product_Name VARCHAR(100),
    Price DECIMAL(10,2),
    Quantity INT
);
```

Expected Output:

```
Query OK, 0 rows affected
```

6. Create a Customer table with Customer_ID, Customer_Name, Email and Phone.

SQL Answer:

```
CREATE TABLE Customer (
    Customer_ID INT,
    Customer_Name VARCHAR(100),
    Email VARCHAR(150),
    Phone VARCHAR(20)
);
```

Expected Output:

```
Query OK, 0 rows affected
```

7. Create a Course table with Course_ID, Course_Name, Duration and Fees.

SQL Answer:

```
CREATE TABLE Course (
    Course_ID INT,
    Course_Name VARCHAR(100),
    Duration VARCHAR(50),
    Fees DECIMAL(10,2)
);
```

Expected Output:

```
Query OK, 0 rows affected
```

8. Create a Faculty table with Faculty_ID, Faculty_Name, Subject and Salary.

SQL Answer:

```
CREATE TABLE Faculty (
    Faculty_ID INT,
    Faculty_Name VARCHAR(100),
    Subject VARCHAR(100),
    Salary DECIMAL(10,2)
);
```

Expected Output:

```
Query OK, 0 rows affected
```

9. Create a Library table with Book_ID, Book_Name, Author and Price.

SQL Answer:

```
CREATE TABLE Library (
    Book_ID INT,
    Book_Name VARCHAR(150),
    Author VARCHAR(100),
    Price DECIMAL(10,2)
);
```

Expected Output:

```
Query OK, 0 rows affected
```

10. Create a Department table with Dept_ID as the Primary Key.

SQL Answer:

```
CREATE TABLE Department (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(100),
    Location VARCHAR(100)
);
```

Expected Output:

```
Query OK, 0 rows affected
```

Note: Use this as an alternative definition of Department if task 4 has already been executed.

11. Create a Student table with Student_ID as the Primary Key and Name as NOT NULL.

SQL Answer:

```
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Age INT,
    Course VARCHAR(100),
    City VARCHAR(100)
);
```

Expected Output:

```
Query OK, 0 rows affected
```

Note: Use this definition instead of the basic Student table in task 2 when demonstrating constraints.

12. Create an Employee table with a UNIQUE constraint on Email.

SQL Answer:

```
CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,
    Emp_Name VARCHAR(100),
    Department VARCHAR(100),
    Email VARCHAR(150) UNIQUE,
    Salary DECIMAL(10,2)
);
```

Expected Output:

```
Query OK, 0 rows affected
```

13. Create a Product table with a CHECK constraint that Price > 0.

SQL Answer:

```
CREATE TABLE Product (
    Product_ID INT PRIMARY KEY,
    Product_Name VARCHAR(100),
    Price DECIMAL(10,2) CHECK (Price > 0),
    Quantity INT
);
```

Expected Output:

```
Query OK, 0 rows affected
```

ALTER TABLE Operations

The following examples assume the Student/Employee tables exist.

14. Add a new Email column to the Student table using ALTER TABLE.

SQL Answer:

```
ALTER TABLE Student
ADD COLUMN Email VARCHAR(150);
```

Expected Output:

```
Query OK, 0 rows affected
```

15. Add a Phone column to the Student table.

SQL Answer:

```
ALTER TABLE Student
ADD COLUMN Phone VARCHAR(20);
```

Expected Output:

```
Query OK, 0 rows affected
```

16. Modify the datatype/size of the Name column in the Student table.

SQL Answer:

```
ALTER TABLE Student
MODIFY COLUMN Name VARCHAR(150) NOT NULL;
```

Expected Output:

```
Query OK, 0 rows affected
```

Note: MySQL uses MODIFY COLUMN to change the datatype/size.

17. Rename the Student table to Student_Details.

SQL Answer:

```
ALTER TABLE Student
RENAME TO Student_Details;
```

Expected Output:

```
Query OK, 0 rows affected
```

18. Rename the column City to Address.

SQL Answer:

```
ALTER TABLE Student_Details
RENAME COLUMN City TO Address;
```

Expected Output:

```
Query OK, 0 rows affected
```

Note: If task 18 is performed before task 17, use Student instead of Student_Details.

19. Add a Salary column to the Employee table.

SQL Answer:

```
ALTER TABLE Employee
ADD COLUMN Salary DECIMAL(10,2);
```

Expected Output:

```
Query OK, 0 rows affected
```

Note: If Salary already exists in the Employee definition from task 3 or 12, this command will produce a duplicate-column error. In that case, skip this demonstration or use a differently named column.

20. Drop the Phone column from the Student table.

SQL Answer:

```
ALTER TABLE Student
```

```
DROP COLUMN Phone;
```

Expected Output:

```
Query OK, 0 rows affected
```

Note: If task 17 has already been completed, replace Student with Student_Details.

21. Drop the Email column from the Student table.

SQL Answer:

```
ALTER TABLE Student
DROP COLUMN Email;
```

Expected Output:

```
Query OK, 0 rows affected
```

Note: If task 17 has already been completed, replace Student with Student_Details.

22. Remove all records from a table using TRUNCATE.

SQL Answer:

```
TRUNCATE TABLE Student;
```

Expected Output:

```
Query OK, 0 rows affected

Student table is now empty.
```

23. Drop the Course table from the database.

SQL Answer:

```
DROP TABLE Course;
```

Expected Output:

```
Query OK, 0 rows affected

Course table removed.
```

24. Drop the Customer table from the database.

SQL Answer:

```
DROP TABLE Customer;
```

Expected Output:

```
Query OK, 0 rows affected

Customer table removed.
```

Complete Student Table + Constraints

Task 25: Primary Key, NOT NULL, UNIQUE and CHECK, followed by ALTER and DROP examples.

25. Create a complete Student table using Primary Key, NOT NULL, UNIQUE and CHECK constraints, and demonstrate suitable ALTER and DROP operations.

SQL Answer:

```
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Age INT CHECK (Age >= 16),
    Course VARCHAR(100) NOT NULL,
    City VARCHAR(100),
    Email VARCHAR(150) UNIQUE
);

-- Demonstrate ALTER: add Phone
ALTER TABLE Student
ADD COLUMN Phone VARCHAR(20);

-- Demonstrate ALTER: rename City to Address
ALTER TABLE Student
RENAME COLUMN City TO Address;

-- Demonstrate ALTER: modify Name size
ALTER TABLE Student
MODIFY COLUMN Name VARCHAR(150) NOT NULL;

-- Demonstrate DROP: remove Phone
ALTER TABLE Student
DROP COLUMN Phone;
```

Expected Output:

```
Query OK, 0 rows affected
Query OK, 0 rows affected
Query OK, 0 rows affected
Query OK, 0 rows affected

Final Student structure can be checked with:
DESCRIBE Student;
```

Note: CHECK prevents invalid ages below 16; NOT NULL requires Name and Course; UNIQUE prevents duplicate Email values; PRIMARY KEY uniquely identifies each student.

Sample final output for Task 25

```
DESCRIBE Student;

+------------+---------------+------+-----+---------+-------+
| Field      | Type          | Null | Key | Default | Extra |
+------------+---------------+------+-----+---------+-------+
| Student_ID | int           | NO   | PRI | NULL    |       |
| Name       | varchar(150)  | NO   |     | NULL    |       |
| Age        | int           | YES  |     | NULL    |       |
| Course     | varchar(100)  | NO   |     | NULL    |       |
| Address    | varchar(100)  | YES  |     | NULL    |       |
| Email      | varchar(150)  | YES  | UNI | NULL    |       |
+------------+---------------+------+-----+---------+-------+
6 rows in set
```

Useful Verification Commands

```
SHOW DATABASES;
USE CollegeDB;
SHOW TABLES;
DESCRIBE Student;
DESCRIBE Employee;
DESCRIBE Department;
```

```
DESCRIBE Product;
```

Typical output:

```
SHOW TABLES;
+--------------------+
| Tables_in_CollegeDB|
+--------------------+
| Department         |
| Employee           |
| Faculty            |
| Library            |
| Product            |
| Student            |
+--------------------+

DESCRIBE Student;
Displays the columns, data types, NULL status, keys and defaults.
```

Important: Tasks 2–13 intentionally include several alternative CREATE TABLE definitions for the same table names. Do not execute every CREATE statement consecutively without dropping/recreating the tables; instead, use each as an individual lab demonstration.